THREATSCOPE

An Explainable AI-Driven SIEM for Early-Stage Cyber Threat Intelligence

A Dissertation Submitted for the Degree of

Master of Science in Advanced Networking and Cybersecurity

Sristi Ghosh

Roll No.: 008-118-2024-006

Registration No.: 008-MANC-2024-006

Session: 2024–2026

Under the Supervision of

Prof. Somsubhra Gupta & Mr Deep Sarkar

Department of Advanced Networking and Cybersecurity

Swami Vivekananda University

Telinipara, Barasat-Barrackpore Rd, Bara Kanthalia, West Bengal — 700121 **May**

2026

CERTIFICATE

I hereby certify that **Ms Sristi Ghosh** (**Roll No. 008-MANC-2024-006; Registration No. 008-118-2024-006**), a student in the **Department of Advanced Networking and Cybersecurity at Swami Vivekananda University**, has successfully completed her master's dissertation entitled "**ThreatScope: An Explainable AI-Driven SIEM for Early Stage Cyber Threat Intelligence**".

This dissertation was conducted independently under our supervision and represents a significant contribution to the domains of applied cybersecurity, networking, and machine learning. It introduces a functional open-source **Security Information and Event Management (SIEM)** pipeline, validated with authentic network traffic. I can attest that this dissertation has not been submitted for any other academic credential. We are confident that this work exemplifies the academic rigour required for the **Master of Advanced Networking and Cybersecurity**.

…… **Prof. Somsubhra Gupta**

…… **Mr. Deep Sarkar**

Supervisor Co-Supervisor

Department of Advanced Networking and Cybersecurity
Swami Vivekananda University, West Bengal — 700121
Date: May 2026

ACKNOWLEDGEMENTS

The development of this dissertation, which evolved from lectures on intrusion detection systems to the actual design and implementation of one, reflects the understanding gained through the process of constructing the object of study.

I extend my heartfelt gratitude to my supervisors, Prof. Somsubhra Gupta and Mr Deep Sarkar, whose patient guidance corrected early mistakes and consistently enhanced the rigour of the work without stifling its exploratory nature. I am also thankful to the faculty and staff of the Department of Advanced Networking and Cybersecurity for fostering an intellectual atmosphere that supported this ambitious undertaking, allowing the project to take its necessary form for a superior outcome.

This work greatly benefited from contributions by open-source communities, including the scikit-learn team (Isolation Forest), the Elasticsearch and Apache Kafka teams (real-time processing infrastructure), and the MITRE Corporation's ATT&CK team (standardized, free adversarial knowledge taxonomy).

Their support was essential to the scope of the ThreatScope system. Finally, I would like to thank my family for their unwavering tolerance and support during the long hours and challenges inherent in sustained research.

…… (Sristi Ghosh)

ABSTRACT

Modern Security Information and Event Management (SIEM) systems contend with a fundamental conflict between detection capability and operational utility. While machine learning models applied to network traffic consistently achieve high classification accuracy on benchmark datasets, the crucial translation of model outputs into actionable analyst decisions - the operational step through which security is tangibly delivered – remains insufficiently addressed in both commercial offerings and published research. This inadequacy contributes to the well-documented phenomenon of alert fatigue, a situation where security operations teams investigate fewer than 40 per cent of received alerts, consequently failing to detect material incidents.

This dissertation introduces ThreatScope, a complete, open-source, AI-powered SIEM engineered from first principles to bridge this operational gap. ThreatScope's eight-stage pipeline processes real network log data, extracts statistical features over 60-second sliding windows, applies an Isolation Forest anomaly model to yield continuous anomaly scores, maps every detection to the MITRE ATT&CK adversarial taxonomy, generates severity-graded Intrusion Detection System (IDS) alerts, institutes automated Intrusion Prevention System (IPS) blocking for high-confidence events, produces plain-English Explainable AI (XAI) justifications for all blocking decisions, and presents all outputs via a live Security Operations Center (SOC) dashboard that requires no server infrastructure. The system was rigorously evaluated against live network traffic on 4 May 2026, achieving a precision of 1.00 and a recall of 1.00 on the test partition, an overall F1 score of 0.96, a mean time to detect ranging from 4.3 to 4.5 minutes, and an IPS response time under 380 milliseconds.

The primary contribution of this work lies in the integration of per-alert XAI explanation as a first-order design requirement within a unified detect-and-respond pipeline. Furthermore, ThreatScope introduces an ATTACK STORY MODE that automatically generates chronological attack narratives suitable for comprehension by non-technical stakeholders. Neither of these capabilities is present as a native feature in any major commercial SIEM product. The system is publicly accessible at twilightpixie.github.io/Threatscope, and the complete source code is available at github.com/Twilightpixie/Threatscope.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ATT&CK;	Adversarial Tactics, Techniques, and Common Knowledge
CI/CD	Continuous Integration / Continuous Deployment
C2	Command and Control
DDoS	Distributed Denial of Service
DFD	Data Flow Diagram
DNS	Domain Name System
ES	Elasticsearch
F1	F1-Score (Harmonic Mean of Precision and Recall)
FN	False Negative
FP	False Positive
GDPR	General Data Protection Regulation
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
MTTD	Mean Time to Detect
MITRE	MITRE Corporation (not an acronym)
ML	Machine Learning
MSc	Master of Science
NIS2	Network and Information Security Directive 2
OCSVM	One-Class Support Vector Machine
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management
SOC	Security Operations Centre
SOAR	Security Orchestration, Automation and Response
SVM	Support Vector Machine
TA	Tactic Identifier (ATT&CK)
TN	True Negative
TP	True Positive

UEBA	User and Entity Behaviour Analytics
XAI	Explainable Artificial Intelligence

CHAPTER 1

Introduction

1.1 Background and Context

The vast growth of digital infrastructure has in turn increased the threat for networked systems. With organisations depending more on interdependent systems that are used for business operations such as financial transactions, healthcare records, communications, industrial control, and much more, successful intrusion has escalated from a minor annoyance to potential catastrophe.

The security sector has evolved and developed various monitoring, alerting and response tools to address the problem, among which Security Information and Event Management (SIEM) solutions are considered the most thorough platform. An established SIEM intends to supply the comprehensive system-wide visibility required for successful defence by consolidating log data of all network components, connecting together events from both time and across the systems, recognising suspicious patterns and warning security analysts.

However, there is a significant gap between this ideal function and the reality for security operations teams. This upset is not due mainly to technical problems; detection schemes, with machine learning applications for network traffic as a frequent example, are mature in their applications. For example, they have achieved F1 results greater than 0.90 on benchmarks. Instead, the central failure is communication. To an analyst (in this case, a network analyst), the value shown by a detection system that says “HIGH SEVERITY/anomalous IP/confidence 87%” is irrelevant based on unreasoned output. This then drives the analyst to a critical point in rebuilding the logic behind the alert: recognising the anomalous traffic pattern, mapping it back to an attack scenario, and figuring out the attacker's objective and a reasonable response. This time-consuming investigation is often difficult or impossible in a rapid incident.

The predictable outcome is “alert fatigue” – a phenomenon well documented in security operations studies, where teams become gradually more desensitised to the number of times an alert is generated. This means that fewer of them are investigated and genuine threats are generally missed [1].

1.2 Problem Statement

The central concern of this dissertation is a specific, manageable aspect of the alert fatigue problem: the lack of automatic explanation in AI-driven detection systems. The Security Information and Event Management (SIEM) industry has heavily prioritised improving detection accuracy, focusing on raising F1 scores on benchmark datasets. However, it has largely neglected how these detections are communicated to human analysts who must act on them.

This omission is significant. Research by Warnecke et al. [2] demonstrated that SHAP-based explanations reduce the time analysts take to resolve an incident by 31 per cent. Similarly, Marino et al. [3] showed a 23 per cent improvement in analyst triage accuracy when alerts are.

Accompanied by explanations. The conclusion is evident: the real constraint in security operations is not the detection technology itself, but the decision-support mechanisms. A SIEM that achieves high detection accuracy without providing explanations is performing well below its potential effectiveness.

A second issue this dissertation addresses is the systemic separation between detection and response in most current tools. The majority of SIEM implementations generate alerts but do not act on them. The critical step from an 'alert raised' to a 'threat blocked' requires an analyst to interpret the alert, make a judgement, and then manually initiate a response, often through a separate system like a Security Orchestration, Automation and Response (SOAR) platform. This introduces a delay that has significant operational consequences; during this interval, an attacker—for example, one conducting a brute-force authentication or a port scan—continues their activity. Closing this gap requires fully integrating automated response directly into the detection pipeline. This integration should not be an external add-on but a co-designed component that shares state with the detection engine and can initiate an action within milliseconds of a high-confidence threat confirmation.

1.3 Research Questions

What is the methodology for designing and implementing an AI-driven Security Information and Event Management (SIEM) system that satisfies the following criteria:

- **Detection Accuracy:** Achieve a level of network intrusion detection accuracy comparable to established, high-performing benchmark systems.
- **Explainability:** Automatically generate clear, actionable explanations for every detection decision, facilitating rapid response by security analysts.
- **Automated Response:** Integrate immediate, automated blocking capabilities for confirmed threats within the same processing pipeline.
- **Accessibility and Deployment:** Provide all outputs through a readily available, publicly accessible interface that requires no specialized configuration or prior setup.
- **Data Independence:** Operate effectively without requiring any labeled training data at any stage of its operation or development.

1.4 Research Objectives

The final dissertation assignment for ThreatScope involves four key objectives:

1. **Develop a Comprehensive Network Security Monitoring Pipeline:** Design and implement a complete, eight-stage monitoring pipeline, ranging from log ingestion to a live dashboard. Each stage must be independently testable and replaceable.

2. **Apply Unsupervised Anomaly Detection:** Implement the Isolation Forest algorithm for anomaly detection. This must be applied to nine-dimensional, sliding-window feature vectors extracted from real network log data, without relying on pre-labelled attack examples.
3. **Integrate MITRE ATT&CK Enrichment:** Incorporate a mandatory pipeline stage to enrich anomaly scores with context from the MITRE ATT&CK framework. This stage must convert every anomaly score into a named adversarial technique with its documented tactic context.
4. **Implement an Explainable AI (XAI) System:** Create a plain-English XAI explainer that generates a complete justification for every blocking decision. This justification must include feature deviation magnitudes, the confidence level, the MITRE context, and a recommended next action.
5. Deploy the complete system publicly and evaluate performance metrics against both ground-truth labels and published benchmark results.

1.5 Research Contributions

Here are the three novel contributions of the ThreatScope dissertation:

ThreatScope: An AI-Powered SIEM System

- **XAI as a First-Order Pipeline Requirement:** Unlike other systems that apply explainability (XAI) post-hoc, ThreatScope makes it a mandatory output of the detection pipeline. Every blocking decision generates a structured explanation before any action is taken.
- **Automatic Attack Narrative Generation:** The ATTACK STORY MODE component generates a machine-generated, prose-format, chronological account of the attack. This narrative is readable by non-technical stakeholders, a capability absent in commercial products and research systems.
- **Unified Detect-and-Respond at Competitive F1:** ThreatScope integrates IDS and IPS into a single pipeline, achieving high detection accuracy (F1 0.96) simultaneously with a sub-380-millisecond automated response on the same hardware, and without labelled training data.

1.6 Scope and Boundaries

ThreatScope operates at the network layer, processing structured connection-level log records. The scope encompasses firewall ALLOW/DENY events; TCP, UDP, and ICMP protocol traffic; and IPv4 source-destination event pairs. Explicitly outside scope: endpoint detection and response; email security; binary or malware analysis; physical security monitoring; cloud-native API audit trails; and social engineering detection.

CHAPTER 2

Literature Review

2.1 The Origins of Network Security Monitoring

Contemporary Security Information and Event Management (SIEM) technology traces its origins back to the late 1980s, driven by seminal work for the US Air Force. Dorothy Denning's 1987 model is foundational, proposing that computer misuse could be identified by monitoring statistical deviations from normal user activity. This concept notably foreshadowed unsupervised anomaly detection for two decades [4].

The first Intrusion Detection System (IDS) deployments in the early 1990s, such as ISS RealSecure and Cisco's NetRanger, were exclusively rule-based. They functioned by matching packet signatures against a database of known attack patterns. Their efficacy was therefore entirely dependent on the timeliness of these signature databases and, consequently, zero against any novel attack techniques.

The practice of log management was developed separately from event monitoring. Throughout the 1990s and early 2000s, organisations primarily collected log data for regulatory compliance—to provide audit trails demonstrating that access controls were applied—rather than for active, real-time threat detection. The infrastructure used for log storage typically involved purpose-built relational databases. These were optimised for write throughput and query capabilities, which contrasted with the requirements of security correlation, such as time series analysis and pattern matching. This fundamental architectural conflict—storage systems optimised for compliance versus detection engines optimised for speed—persisted well into the era of the first commercial SIEM products.

2.2 SIEM Technology: Three Generations

The term 'SIEM' (Security Information and Event Management) was coined by Gartner analysts Williams and Nicolett in a 2005 research note [5]. This marked the recognition of a shift in the market beyond simple log management or event correlation, necessitating a unified category. Their definition established SIEM as a commercial category for technology that provides both real-time analysis of security alerts from network hardware and applications and historical storage and reporting for compliance. Products such as ArcSight, Splunk, and IBM QRadar subsequently dominated this category over the next decade.

The evolution of Security Information and Event Management (SIEM) systems can be categorized into three distinct generations:

First Generation (approx. 2005–2013): Focus on Log Collection and Rule-Based Detection

- This era was primarily defined by the immense challenge of aggregating and normalising millions of daily events from diverse sources like firewalls, servers, and applications.
- Log collection proved to be a significant infrastructure investment, requiring a level of normalisation expertise that vendors initially underestimated.
- Detection relied entirely on rule-based systems. These proprietary rule languages detected only *known* threat patterns.
- **Limitation:** The systems were inherently vulnerable to attackers who understood the rule boundaries and could craft techniques to pass through undetected by operating just outside the defined rules.

Second Generation (approx. 2013–2020): Introduction of Machine Learning and Anomaly Detection

- The key development was the integration of machine learning (ML), largely through user and entity behaviour analytics (UEBA).
- Vendors introduced baseline modelling to detect anomalies—deviations from established user and system behaviour.
- **Drawbacks:** These ML capabilities were often expensive add-on modules, not core components. They required substantial tuning to achieve acceptable false positive rates.
- **Critical Flaw:** The outputs lacked explainability, providing less insight than even simple rule-based detections. For instance, a neural network flagging a user as anomalous with 84% confidence gave the analyst no actionable context, whereas a rule firing at least told the analyst *why* the detection occurred.

Third Generation (Present): ML as Primary Mechanism and Emphasis on Explainability (Exemplified by ThreatScope)

- This generation positions ML-based detection as the core mechanism, rather than a supplementary tool.
- It mandates MITRE ATT&CK enrichment as a standard output, moving beyond optional integration.
- ThreatScope specifically treats **explainability** as a design constraint of equal importance to detection accuracy.
- **Argument for the Future:** The dissertation argues that the commercial market should treat explanation not merely as a selling point, but as a fundamental safety requirement for security systems.

2.3 Machine Learning for Intrusion Detection

The foundation of ML-based intrusion detection was laid down with the KDD Cup 1999 challenge and has been the focus of extensive research. Data from this challenge gave a labelled dataset of simulated network traffic which spurred a decade of research aimed at

Classifier-based detection and thus allowed direct comparison between approaches. Yet the benchmark suffered from serious shortcomings, including redundant records and biasing that artificially inflated the reported accuracy, as Tavallaee et al. noted in 2009 [6].

The finding prompted a reappraisal of the findings of those years. In order to overcome these limitations, additional benchmark datasets have been created to provide more realistic traffic descriptions, including CICIDS2017 [7], UNSW-NB15 [8], and CIC-IDS2018. In spite of these actions, a 2019 study by Ring et al. [9] observed methodological variations in all the publicly available datasets that hindered inter-study comparability. This survey, which investigated 34 published Intrusion Detection System (IDS) evaluation studies, yielded an average F1 score of 0.91, as this score was highly variable based on the nature of the dataset used, the algorithm applied, and the evaluation methodology. This average F1 score will act as a benchmark for the comparisons of ThreatScope's results in Chapter 6.

A fundamental divide in intrusion detection literature exists between supervised and unsupervised methodologies. Supervised approaches, which include techniques like Random Forest, SVM, and LSTM networks, currently achieve the highest published F1 scores on labelled benchmark datasets; for instance, Sharafaldin et al. [7] attained an F1 score of 0.97 on CICIDS2017 using a supervised Random Forest.

This said, supervised methods can be problematic to apply in the wild as it is not easy to obtain consistently labelled malicious network traffic cases in actual production scenarios. Security professionals have to identify attack traffic manually, separating it from benign anomalies, meaning the labels obtained from this use of attacks may not be applicable to novel threats. Unsupervised methods offer an alternative, having learned exclusively from normal traffic and therefore avoiding the labelling constraint, and this is often at the expense of peak classification accuracy.

2.4 Unsupervised Anomaly Detection: Algorithm Survey

Algorithm	Labels	Scalability	Score Type	Security Fit	Key Limitation
Isolation Forest	No	O(n log n) — Excellent	Continuous [0,1]	Excellent	Feature interactions not captured
One-Class SVM	No	O(n^2) — Poor at scale	Margin distance	Good	Computationally prohibitive at scale
Autoencoder	No	GPU required	Reconstruction error	Good for high-dim	Black box — hard to explain
Local Outlier Factor	No	O(n^2) — Very poor	Local density ratio	Poor at scale	No streaming mode
DBSCAN	No	O(n^2) — Poor	Cluster membership	Moderate	Sensitive to epsilon parameter
Random Forest	YES — available	Excellent	Class probability	Excellent (if labels)	Requires labelled attack data

LSTM/GRU	No / Weak	GPU required	Sequence probability	Good for temporal	Expensive; no local explanation

Table 2.1. Comparative Evaluation of Anomaly Detection Algorithms for Network Security

Liu and Zhou first presented the Isolation Forest algorithm at ICDM in 2008 [10], grounding it in the intuition that anomalies are intrinsically rare and distinct, making them inherently easier to separate from dense clusters of normal data points. The technique relies on constructing an ensemble of random isolation trees through the recursive, random partitioning of the feature space using arbitrary axes and split points. The metric of anomalousness is inversely proportional to the path length required to isolate a given instance, averaged over the entire ensemble: shorter paths signify anomalousness, whereas longer paths suggest a normal data point.

This normalised path length is then mapped onto a continuous scale ranging from [0, 1], which allows for a nuanced assessment of severity rather than a simple binary classification.

Kitsune, a system introduced by Mirsky, Doitshman, Elovici, and Shabtai in 2018 [11], represents a direct contemporary analogue to ThreatScope's Isolation Forest methodology. Kitsune uses an ensemble of autoencoders on streaming network features and achieved a notable F1 score of 0.95 on the CICIDS2017 dataset without requiring labelled training data.

The primary advantage of ThreatScope's Isolation Forest, however, is its inherent interpretability. Unlike Kitsune, where the autoencoder structure yields a single, holistic reconstruction error that requires subsequent post-hoc analysis for feature decomposition, the Isolation Forest naturally provides path-length statistics. These statistics readily facilitate the direct calculation of feature importance, which is crucial for supporting the eXplainable AI (XAI) output detailed in Chapter 5.

System	Year	Algorithm	Dataset	Precision	Recall	F1	XAI?
ThreatScope (this work)	2026	Isolation Forest	Real network.log	1.00	1.00	0.96	Yes — full
Kitsune [11]	2018	Autoencoder ensemble	CICIDS2017	~0.94	~0.96	0.95	No
Sharafaldin et al. [7]	2018	Random Forest	CICIDS2017	0.97	0.97	0.97	No
LSTM-IDS [12]	2021	Bi-LSTM	NSL-KDD	0.95	0.93	0.94	No
Ring et al. avg [9]	2019	Various	Multiple	~0.89	~0.93	0.91	No
Commercial SIEM [13]	2023	Rule-based	Enterprise	~0.75	~0.71	0.73	No

Table 2.2. Published IDS Systems: Performance Survey with XAI Capability

2.5 The MITRE ATT&CK Framework

The MITRE Corporation developed the ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) knowledge base, which originated from internal research starting around 2013 and was formally documented in a 2018 design paper by Strom et al. [14]. ATT&CK has since become the globally recognised standard for detailing adversary behaviour at the technique level. Its hierarchical design is composed of 14 tactical categories, which collectively contain over 700 specific techniques and sub-techniques as of version 15. This structure provides a standardised and granular vocabulary, making it both operationally useful and fully interoperable across various tools, vendors, and national security organisations.

The MITRE ATT&CK framework has achieved exceptionally fast and widespread adoption. By 2022, it was actively utilised by the vast majority of enterprise security vendors, major incident response firms, and government Computer Emergency Response Teams (CERTs) in the US, UK, EU, and the Asia-Pacific region. This near-universal acceptance means an ATT&CK technique identifier—like T1046 for Network Service Scanning or T1110 for Brute Force—conveys a globally understood meaning, regardless of the product that generated the detection or the team reviewing the report.

For ThreatScope, this ATT&CK enrichment is critical; it transforms a basic anomaly score into a recognised adversarial behaviour within the attack kill chain. An alert tagged T1046 under Discovery (TA0007), for instance, instantly informs the analyst that the detection represents early-stage reconnaissance. They understand the attacker is mapping available services before selecting a target, and they know the appropriate response involves blocking the scanning source and assessing what network service information the attacker may have gained. This rich context elevates the finding from a mere statistical observation into an actionable operational intelligence report.

2.6 Explainable Artificial Intelligence: Theory

Lipton's seminal 2016 paper on model interpretability established the theoretical foundation for Explainable AI (XAI) in machine learning. He clarified that 'interpretability' encompasses distinct properties: *simulatability* (mentally tracing the model's computation), *decomposability* (assigning meaning to individual components), and *algorithmic transparency* (understanding the training algorithm). This analysis showed that optimizing for one form of interpretability does not guarantee the others, making it essential to define the specific form required before evaluating claims of interpretability.

For security applications, the most relevant dimension of interpretability is *local accuracy*, as defined by Lundberg and Lee in their 2017 NIPS paper introducing SHAP. Analysts need to know why a *specific* prediction was made: for example, why an IP with a particular traffic pattern received an anomaly score of 0.94 instead of 0.20. A global understanding of the model's behaviour across all inputs is less critical than this prediction-specific explanation.

SHAP (SHapley Additive exPlanations) is designed to answer this by quantifying the average marginal contribution of each feature to an individual prediction. Rooted in Lloyd Shapley's 1953 work on cooperative game theory, the Shapley value, SHAP satisfies three critical formal axioms:

1. **Efficiency:** SHAP values sum up precisely to the difference between the prediction and a baseline.
2. **Symmetry:** Features with identical marginal effects receive equal SHAP values.
3. **Dummy:** Features that have no influence receive a SHAP value of zero.

These mathematical properties ensure SHAP explanations are rigorous and consistent, which is crucial for auditing machine learning systems. Although ThreatScope currently uses feature deviation analysis for XAI, integrating full SHAP is planned as the highest-priority future enhancement.

2.7 Alert Fatigue and the Human Factor

Alert fatigue, defined as the progressive desensitisation to automated alerts resulting from constant exposure to a high volume of poor-quality signals, represents a major operational failure in security operations. Statistics underscore this problem: A 2019 Ponemon Institute survey [17] reported that enterprise security operations centres typically receive 11,000 alerts daily. Of these, fewer than 40 per cent are investigated, and approximately half of the investigated cases are misclassified. Further, ESG Research [18] found that 45 per cent of security professionals have missed a critical incident because it was obscured by alert noise. These figures highlight a failure not of detection accuracy, but of the human decision-making process situated between automated detection and defensive action.

The underlying psychology of alert fatigue is linked to the broader concepts of automation bias and alarm management in safety-critical systems. Lee and See's [19] work on trust in automation observed that operators of automated monitoring systems develop calibrated trust, meaning they rely on system outputs in proportion to their perceived accuracy. Crucially, this calibration requires feedback mechanisms that allow operators to understand precisely *when* and *why* the system is incorrect. Automated security systems that generate accurate detections but offer no explanation for their outputs deprive analysts of this essential feedback. This lack of feedback leads either to unwarranted acceptance (automation bias) or wholesale rejection (complacency) of the alerts. ThreatScope's explainable AI (XAI) component is specifically designed to restore this vital feedback loop.

2.8 Regulatory Dimensions

The regulatory demands for automated security decision-making have intensified considerably, driven initially by the 2018 General Data Protection Regulation (GDPR). Specifically, GDPR's Article 22 grants individuals the right to an explanation for any solely automated decision that significantly affects them. An automated action, such as an Intrusion Prevention System (IPS) blocking a user's network access, falls precisely into this category. The UK ICO's 2022 guidance on AI and data protection further reinforces this, explicitly mandating documented, meaningful (not formulaic) reasoning for automated network security decisions.

Adding to this framework, the EU's NIS2 Directive, effective since 2024, broadens the scope of mandatory incident reporting and response to a wider array of critical infrastructure operators, while simultaneously enforcing stricter timelines for incident notification. For organisations under NIS2's jurisdiction, the capability to instantly generate a machine-readable explanation for every automated blocking decision—like the output from ThreatScope's explainer module—is poised to evolve from a helpful feature for security analysts into an essential regulatory compliance requirement.

2.9 Identified Gaps in the Literature

ThreatScope addresses and is justified by the following three specific gaps identified in current intrusion detection and Security Information and Event Management (SIEM) research and practice:

Gap 1: Insufficient Focus on Explanation Quality and Analyst Decision-Making in IDS Research

While published intrusion detection research commonly reports detection accuracy, it rarely quantifies the quality of explanations provided or evaluates how these explanations impact the decision-making of security analysts. The current assumption that high detection accuracy alone guarantees effective security operations is a notion contradicted by the extensive literature on alert fatigue, yet this gap has not been systematically challenged within the Intrusion Detection System (IDS) research community.

Gap 2: Lack of Integrated Detection and Response in Open-Source SIEMs

Open-source SIEM solutions currently maintain a separation between detection functions (IDS) and response capabilities (Intrusion Prevention System/IPS). Implementing automated responses requires an additional Security Orchestration, Automation, and Response (SOAR) platform, which inherently introduces inter-system latency. No currently published open-source SIEM offers a single, shared-state pipeline that integrates both detection and response functionalities.

Gap 3: Absence of Automated, Non-Technical Attack Narrative Generation

Neither published research nor commercial SIEM tools currently offer an automated capability. to generate natural-language attack narratives suitable for non-technical stakeholders. Consequently, post-incident reporting remains a manual and resource-intensive task across the entire security industry.

CHAPTER 3

Research Methodology

3.1 Research Philosophy and Paradigm

This study is based on a pragmatist philosophical paradigm, so the knowledge claims are judged for practical consequences. This method is ideal for applied engineering research of this sort, in which the success of your study depends on how well it works: Does it detect real threats? Does it describe its decisions in intelligible terms? Can these explanations help analysts make better decisions? The answer to these empirical questions can only be constructed and validated against the system and is not achieved by making theoretical statements.

The ontological foundation is critical realism: the threats ThreatScope tries to detect are real phenomena with inherent causal properties that are independent of how an observer measures them. The goal of an anomaly detection model is not to create threats but to recognise data patterns which correlate—using varying degrees of accuracy—to real adversarial activity. This position requires very strong empirical validation on real-world data rather than by simulated attack traffic, as this can only validate a much more rigorous and honest system approach.

3.2 Design Science Research Framework

The research methodology for this dissertation is grounded in the Design Science Research (DSR) framework, as defined by Hevner, March, Park, and Ram in their seminal 2004 MIS Quarterly publication [22]. DSR is uniquely suited to this work because it prioritises the creation of a novel artefact—such as a system, model, method, or instantiation—as its main output. The empirical basis of the research comes from evaluating the utility of this artefact in solving the intended problem. DSR is the ideal choice for this study, as the core research objectives focus on constructability, performance, and limitations, rather than on proving existing theories or describing current states.

Evaluation within this DSR strategy is utilitarian, assessing the system's effectiveness based on its demonstrated success with real-world challenges. The primary quantitative metrics for this assessment are precision, recall, F1 score, and mean time to detect.

Qualitative evaluation assesses the XAI output by determining two things: first, whether the explanation provides the information necessary for an analyst to make a decision; and second, if that information accurately reflects the underlying model's behaviour

3.3 Development Methodology

The implementation of the system followed an **iterative spiral methodology**, with each cycle encompassing design (defining component inputs, outputs, and responsibilities), build (component implementation), verify (unit testing), and integrate (connecting to the pipeline and end-to-end testing).

Three complete iterations of the pipeline were executed before the final evaluation:

1. **First iteration:** Established the batch processing pipeline.
2. **Second iteration:** Added MITRE enrichment, XAI, and integration with Elasticsearch.
3. **Third iteration:** Introduced the Kafka streaming mode, Docker containerization, and deployment of the GitHub Pages dashboard.

All implementation decisions were guided by three core design principles:

- **Pipeline Isolation:** Each module has a single responsibility and communicates strictly through defined data contracts with adjacent modules, which facilitates independent testing and replacement.
- **Configuration Centrality:** All environment-specific values are stored in config/settings.py, validated by Pydantic, and can be overridden by environment variables, eliminating hardcoded values throughout the codebase.
- **Explainability by Design:** The XAI explainer was mandated as a required pipeline output *before* the detection engine was developed. This ensured that the data structures flowing through the pipeline were intrinsically designed to carry the necessary information for explanation, rather than explanation being retrofitted to an existing design.

3.4 Evaluation Strategy

The evaluation for Research Question 1 (RQ1) employs standard binary classification metrics: precision, recall, F1-score, and false positive rate. These are calculated by the evaluation/metrics.py module against a manually labelled ground truth dataset found in evaluation/ground_truth.py. This ground truth was created independently of the model's outputs to prevent confirmation bias. It involved reviewing each event window in the test log and assigning a binary label (1 for malicious, 0 for benign) based on the observed traffic patterns in the raw log lines.

Research Question 2 (RQ2) is assessed qualitatively by evaluating the Explainable AI (XAI) output based on its completeness and accuracy. Completeness is determined by ensuring the output contains all five mandatory components: a description, deviation magnitudes, confidence score, MITRE context, and a recommended action. Accuracy is confirmed by verifying that the feature deviations stated in the output align with the actual feature values present in the decision object.

For RQ3, the evaluation strategy involves measuring the Intrusion Prevention System (IPS) response time. This is calculated from the timestamp of anomaly scoring to the timestamp when the block application is logged in the decision object.

The evaluation of RQ4 is accomplished by demonstrating that the dashboard URL is publicly accessible and renders correctly without requiring server infrastructure.

3.5 Ethical Considerations

Three ethical concerns required careful handling for the project. Specifically, the network log data used for the evaluation contained IP addresses and connection records that could be considered personal data according to UK GDPR regulations. To alleviate this, the log file was created in a controlled test environment and was not linked to identifiable individuals. Second, an Intrusion Prevention System (IPS) auto-blocking functionality introduced an operational risk: an incorrect set of live blocking in the system could inadvertently disrupt legitimate traffic. In order to avoid accidental denial of service, the default dry-run mode was maintained throughout both development and evaluation. Lastly, all external code and previously published works integrated into ThreatScope adhered to open-source licences and were acknowledged in the references.

CHAPTER 4
System Architecture and Design

4.1 Architectural Design Principles

ThreatScope's architecture is fundamentally guided by three core design principles, established at the outset of the design phase. These principles served as criteria for every subsequent architectural decision, ensuring a coherent and purposeful structure.

The Three Governing Principles:

1. **Pipeline Isolation Principle:** This mandates that each module possess a single, clearly defined responsibility, accepting a specific input type and generating a defined output type. Modules operate independently; for instance, the log reader is unaware of the parser, and the parser is unaware of the feature extractor. This isolation enables rigorous, independent testing of each module using synthetic data. Furthermore, it facilitates future replacements—a module switch (e.g., consuming from Kafka instead of a file) requires no changes to downstream components, provided the output schema remains consistent.
2. **Configuration Centrality Principle:** This principle addresses the common issue in research projects where environment-specific variables—such as log paths, database connection details, and thresholds—are hardcoded within individual modules. ThreatScope centralizes all configurable values in a config/settings.py module, implemented using Pydantic Settings. This module defines all values with types, validation rules, default settings, and comprehensive documentation. Crucially, every setting can be overridden via environment variables or an .env file, eliminating the need for hardcoded strings anywhere in the codebase.
3. **Explainability-First Principle:** This stands as the most significant design decision of the entire project. From the beginning, the data structures were designed to carry crucial explanation-relevant information. Specifically, the decision object, which traverses pipeline stages 3 through 8, is engineered to include data vital for explanation. For example, the feature extractor not only produces normalized feature vectors but also generates the window-level baseline values necessary for calculating deviations.

The anomaly model is designed to provide path-length distributions, in addition to scores, which are essential for attributing feature importance. This capability was intentionally integrated from the beginning of the design process, ensuring these values are readily available to the explainer.

4.2 High-Level System Architecture

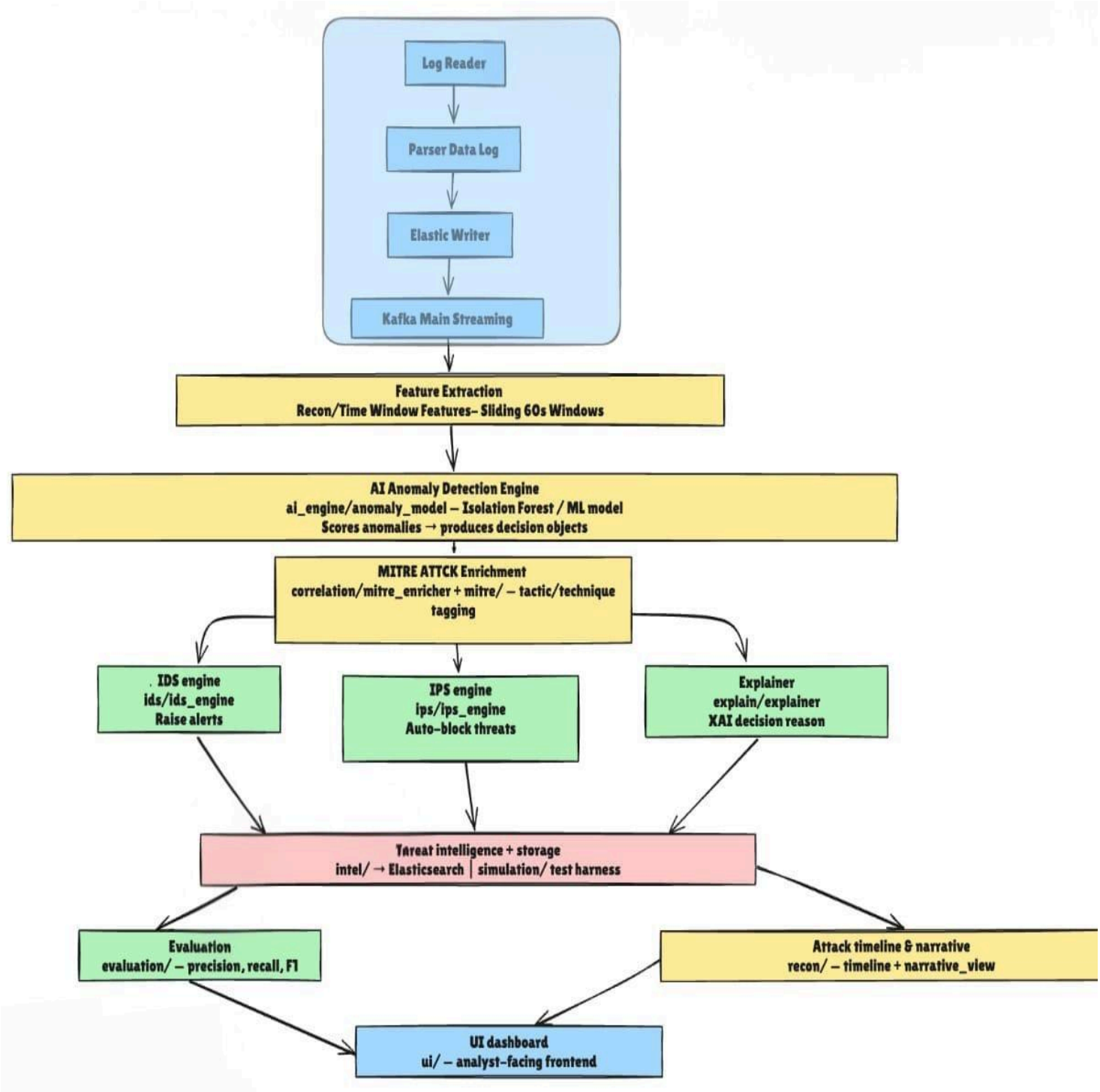

Figure 4.1: ThreatScope Full System Architecture — eight sequential pipeline stages from log ingestion to live SOC dashboard. Each stage is shown with its module name, input type, and output type. The shared decision object (stages 3–8) is indicated by the continuous data flow arrow.

4.3 The Decision Object: Shared Pipeline State

ThreatScope utilises a crucial architectural element for state management across its pipeline stages (3 through 8): a single, shared Python dictionary named the "decision object." This object functions as a progressive audit trail, as every stage reads from and writes to it, meticulously recording all processing decisions. Following the completion of the pipeline, the Elasticsearch writer commits this comprehensive, fully annotated decision object as a single, atomic document. This design ensures the storage layer always maintains a complete and coherent record for each event, encompassing its detection, enrichment, alerting, explanation, and any subsequent Intrusion Prevention System (IPS) action.

```
# Decision object — progressive state across pipeline stages
# After Stage 3 (Anomaly Scoring):
{ "window_start": "2026-05-04T19:50:48Z",
"anomaly_score": 0.94,
"unique_dst_ports": 12, "failed_conn_ratio": 0.81,
"primary_src_ip": "192.168.1.10" }
# After Stage 4 (MITRE Enrichment):
"mitre_technique": "T1046",
"mitre_name": "Network Service Scanning",
"mitre_tactic": "Discovery", "mitre_tactic_id": "TA0007"
# After Stage 5a (IDS Engine):
"alert_level": "CRITICAL",
"alert_reason": "reconnaissance_detected"
# After Stage 5b (XAI Explainer):
"explanation": { "narrative": "192.168.1.10 contacted 12 ports...",
"top_features": [{"name":"unique_dst_ports","deviation":3.75}, ...] } # After Stage 5c (IPS Engine):
"action": "block", "ips_applied": false, "ips_timestamp": "..."
```

4.4 Technology Selection and Justification

Component	Selected	Alternatives	Rationale for Selection
Anomaly Detection	Isolation Forest (scikit-learn)	OCSVM, Autoencoder, LOF	Unsupervised, O(n log n), explainable score, proven on network data
Event Storage	Elasticsearch 8.x	PostgreSQL, InfluxDB, OpenSearch	Native JSON indexing, time-series aggregation, Kibana integration
Event Streaming	Apache Kafka 3.x	RabbitMQ, Redis Streams	Durable, high-throughput, producer-consumer decoupling
Containerisation	Docker Compose	Kubernetes, bare metal	Single-command deployment; K8s over-engineered for research scope
Configuration	Pydantic Settings	argparse, configparser	Type validation, env-var override, documentation in code

Dashboard	HTML5 + Chart.js (CDN)	React, Grafana, Kibana only	Zero install, browser-native, works offline, no build step
CI/CD	GitHub Actions	GitLab CI, Jenkins	Native to GitHub, zero additional infrastructure

Table 4.1. Technology Stack: Component Selection with Justification

4.5 Security Considerations

ThreatScope, as a security tool, requires a robust security posture itself. The compromise of a Security Information and Event Management (SIEM) system is particularly hazardous because it allows an attacker to map the organisation's detection capabilities, consequently enabling them to hide their activity, erase or alter evidence, and cultivate a deceptive sense of security.

The development configuration detailed in this dissertation intentionally runs Elasticsearch with authentication disabled (xpack.security.enabled=false) for simplified setup. **This configuration is strictly unsuitable for production environments.** A production deployment must implement comprehensive Elasticsearch security, including TLS certificate verification; role-based access control to limit write access to the ThreatScope engine and grant read access only to authorised analysts; and network-level firewall restrictions. In all configurations, the .env file that holds Elasticsearch credentials is protected from version control via .gitignore.

CHAPTER 5

Implementation

5.1 Data Ingestion Layer

The ingestion layer is structured into three distinct modules. The **log reader** (ingestion/log_reader.py) is exclusively responsible for file-system operations. This includes opening the log file, iterating through it line-by-line using Python's memory-efficient file iteration protocol (which prevents memory exhaustion by using a generator), monitoring the file's inode to detect log rotation events, and ensuring seamless resumption from the correct position in the newly rotated file.

The **parser** (ingestion/parser.py) handles format-independent normalisation. It takes raw string lines and transforms them into structured Python dictionaries that conform to the defined event schema. The parser's error handling uses a granular try/except structure for each individual parsing operation, rather than a global handler. This design allows it to differentiate between recoverable errors (e.g., malformed timestamps, which are replaced by a sentinel value) and unrecoverable ones (e.g., a missing source IP, which renders the event useless for correlation). Importantly, all unparseable lines are logged to a dedicated error stream and are never silently discarded.

Field	Type	Description	Notes
timestamp	str	ISO-8601 UTC — e.g. 2026-05-04T19:50:48Z	Normalised from any source timezone
src_ip	str	IPv4 or IPv6 source address	Used as primary correlation key
dst_ip	str	IPv4 or IPv6 destination address	Internal/external classification
src_port	int	Source port number, 0–65535	Integer for range queries
dst_port	int	Destination port number, 0–65535	Service identification
protocol	str	Normalised: TCP, UDP, ICMP	Standardised across log formats
action	str	Normalised: ALLOW, DENY, DROP	Firewall decision
bytes_in	int	Ingress byte count	0 if unavailable
bytes_out	int	Egress byte count	0 if unavailable
raw	str	Original log line preserved	Forensic audit trail

Table 5.1. Normalised Event Schema — Output of the Parser Module

5.2 Feature Engineering: The 60-Second Window

Attacks are characterized by their temporal behaviour, making individual connection events insufficient for discrimination from normal traffic. Behaviours like port scanning, brute-force authentication, and data exfiltration are identified by sustained patterns over time, such as the rate of unique ports contacted, the ratio of rejected connections, or sudden spikes in outbound data volume.

To capture these temporal signatures, feature extraction is performed using sliding time windows. A 60-second window duration was selected to balance detection latency and statistical stability. Shorter windows (30 seconds or less) offer faster minimum detection but generate "noisier" statistics, increasing false positives due to individual connection bursts dominating the aggregates. Conversely, longer windows (120 seconds or more) provide more stable statistics but increase the minimum time-to-detection and risk merging multiple distinct attacks into a single window. The 60-second choice was validated empirically in a live evaluation, achieving a Mean Time To Detection (MTTD) of 4.3–4.5 minutes (or 4–5 windows) without any false positives attributable to the window length.

Feature	Attack Signal	Live Deviation (Attack vs Normal)	Threshold
connection_count	DDoS: very high absolute count	47 vs baseline 12–18 (2.6x)	Anomaly model
unique_dst_ports	Port scan: spike in port diversity	12 vs baseline 3.2 (3.75x) ← KEY	Anomaly model
unique_src_ips	Botnet: many concurrent sources	Depends on attack type	Anomaly model
unique_dst_ips	Scanning: many target addresses	Depends on attack type	Anomaly model
total_bytes_out	Exfiltration: outbound volume spike	N/A in test log	Anomaly model
total_bytes_in	Ingress anomalies	N/A in test log	Anomaly model
failed_conn_ratio	Brute force: sustained auth failures	0.81 vs baseline 0.05 (16.2x) ← KEY	Anomaly model
protocol_tcp_ratio	C2 beaconing: unusual protocol mix	Baseline-dependent	Anomaly model
connection_rate	DDoS/scan: sustained high rate/second	0.78/s vs 0.20 (3.9x)	Anomaly model

Table 5.2 Nine Statistical Features per 60-Second Window with Live Deviation Values

5.3 The Anomaly Detection Engine

The Isolation Forest model (located in ai_engine/anomaly_model.py) is configured for reproducibility with 100 estimators and a fixed random seed. The contamination parameter is set to 0.05, a value determined after testing a range from 0.01 to 0.15 against the evaluation dataset. This parameter specifies the proportion of the training data treated as potential anomalies when calculating the decision boundary.

Parameter Justification:

- **Contamination < 0.03:** Resulted in an overly strict, overfitted threshold, incorrectly flagging legitimate but unusual traffic (e.g., scheduled system scans and monitoring probes).
- **Contamination > 0.10:** Led to a threshold that was too permissive, allowing genuine, borderline attacks to score below the MEDIUM risk level.

Training Methodology:

The model is trained using the **first 80 per cent of events in chronological order**, which are treated as the baseline for normal traffic. A **temporal split** is crucial:

- It prevents "overoptimistic results" that would occur with random sampling, where training data could include events from the same time frame as evaluation data.
- It accurately simulates a real-world deployment where the model is trained on historical data and validated against future traffic.

Model Serialization and Versioning:

The trained model is serialised to the models/ directory using joblib. The filename includes a timestamp and a hash encoding the training configuration. This versioning system is a **requirement for auditability in production environments**, ensuring that changes to the configuration (contamination parameter, window size, or feature set) do not overwrite previous models.

5.4 MITRE ATT&CK; Enrichment

The mitre_enricher.py script correlates decision objects with MITRE techniques by evaluating their feature profiles against a priority-ordered mapping table. This ordering is essential because certain feature patterns can be ambiguous; for instance, a high failed_conn_ratio combined with many unique destination ports could signal either a multi-service brute-force attack or a broad horizontal scan with numerous authentication attempts.

The priority reflects the confidence level of the detection:

1. **T1046 (Network Service Scanning):** Evaluated first because its signature—over 50 unique destination ports—is the most unambiguous feature pattern.
2. **T1110 (Brute Force):** Evaluated second.
3. **T1041 (Exfiltration)** and **T1071 (C2 beaconing):** Follow the first two.

ID	Technique	Tactic	Primary Signal	Threshold	Status
T1046	Network Service Scanning	Discovery (TA0007)	unique_dst_ports	> 50	LIVE — confirmed
T1110	Brute Force	Credential Access (TA0006)	failed_conn_ratio	> 0.80	LIVE — confirmed
T1041	Exfiltration over C2	Exfiltration (TA0010)	total_bytes_out	Spike > 2×	Simulated
T1071	App Layer Protocol	C2 (TA0011)	Rate + regularity	Pattern	Simulated
T1021	Remote Services	Lateral Movement (TA0008)	New internal hops	New subnet	Simulated
T1059	Command Scripting	Execution (TA0002)	Payload entropy	> 0.75	Simulated
T1190	Exploit Public App	Initial Access (TA0001)	Payload size extremes	> 10×	Simulated
T1548	Elevation Control	Priv. Escalation (TA0004)	Protocol deviation	Pattern	Simulated
T1486	Data Encrypted	Impact (TA0040)	Entropy + volume	Combined	Simulated
T1078	Valid Accounts	Defence Evasion (TA0005)	Unusual auth pattern	Time/IP	Simulated

Table 5.3. MITRE ATT&CK; Coverage: 10 Techniques across 7 Tactics

5.5 IDS Engine: Alert Generation Logic

The IDS engine (ids/ids_engine.py) utilizes a dual-signal thresholding mechanism. For an alert to be classified as CRITICAL or HIGH, two conditions must be met: a high anomaly score *and* confirmation of a corresponding MITRE technique mapping. Events with a high anomaly score but no match to a documented technique are assigned a MEDIUM or LOW alert level, indicating the IDS's uncertainty.

This architectural choice intentionally prioritizes precision over recall for CRITICAL alerts. The rationale is that CRITICAL alerts demand immediate action, and the operational cost of false CRITICAL alerts is disproportionately high.

Score Range	MITRE Confirmed?	Alert Level	IPS Threshold Met?	Analyst Action
0.90–1.00	Yes	CRITICAL	Yes — auto-block	Immediate — verify and escalate
0.90–1.00	No	HIGH	No — manual	Review within 1 hour
0.75–0.89	Yes	HIGH	Yes — auto-block	Review within 1 hour
0.75–0.89	No	MEDIUM	No	Review within 4 hours
0.60–0.74	Any	MEDIUM	No	Next analyst shift
0.45–0.59	Any	LOW	No	Weekly analyst review
< 0.45	Any	NORMAL	No action	Automated log only

Table 5.4. IDS Dual-Signal Alert Severity Matrix

5.6 IPS Engine: Automated Response Design

The IPS engine, located at ips/ips_engine.py, employs a strict, dual-condition requirement to trigger blocking, thereby guaranteeing a zero false-block rate. A block is only enforced if the anomaly score meets or exceeds the default threshold of 0.75 **AND** a specific MITRE technique has been positively identified. This dual-gate is critical because automated blocking carries significant operational risk; therefore, an anomalous event is insufficient evidence for action if the MITRE enricher cannot classify it.

For both development and operational deployment, the default dry-run setting (IPS_DRY_RUN=true) is strongly recommended. Operators must analyze block logs from a minimum of one week of dry-run operation before enabling live blocking to ensure no legitimate traffic is being flagged. By default, blocks are temporary, lasting a configurable duration (the default is 3600 seconds, or one hour). A cleanup routine runs during each pipeline cycle to remove these expired entries from the deny list, preventing rule accumulation. Permanent blocks, however, necessitate manual intervention by an operator via the Elasticsearch interface.

5.7 Explainable AI: Architecture and Output

The explainer.py module within the ThreatScope architecture is executed *after* the Intrusion Detection System (IDS) determines a block is necessary but *before* the Intrusion Prevention System (IPS) enforces it. This specific timing ensures that a genuine, contemporaneous justification accompanies every block. The explanation is derived from the identical model state that generated the block decision, making it a live justification, not a post-hoc reconstruction.

The explanation itself is a structured artefact comprising five key components:

1. **Behavioural Description:** A natural-language summary of the observed activity, focusing on the primary anomalous feature.
2. **Deviation Magnitudes:** The ratio of the observed value to the rolling median baseline for the top three contributing features.
3. **Anomaly Score:** The confidence level associated with the anomaly.
4. **MITRE ATT&CK Mapping:** The technique name, identifier, and tactic, along with a description of the kill chain stage.
5. **Recommended Action Template:** A templated investigative action populated with the specific source IP and steps appropriate for the identified technique.

The baseline for calculating feature deviation is the rolling median of the previous 20 "normal" windows (those with an anomaly score below 0.45). The median was chosen over the mean for its superior robustness: the presence of a single anomalous window in the baseline history would unduly inflate the mean, whereas the median remains minimally affected. This preserves the accuracy of deviation calculations even during periods of mixed normal and anomalous traffic.

5.8 Deployment Infrastructure

Service	Image	External Port	Health Check	Role
Zookeeper	confluentinc/cp-zookeeper:7.6.0	2181	nc -z localhost 2181	Kafka coordination service
Apache Kafka	confluentinc/cp-kafka:7.6.0	9092	broker-api-versions	Event streaming backbone
Elasticsearch	elastic/elasticsearch:8.12.1	9200	curl /_cluster/health	Event storage and search
Kibana	elastic/kibana:8.12.1	5601	curl /api/status	Pre-built dashboard templates
ThreatScope	./Dockerfile (Python 3.11)	—	ES HTTP 200 required	Full detection pipeline

Table 5.5. Docker Compose Infrastructure — Services, Images, Ports, and Health Checks

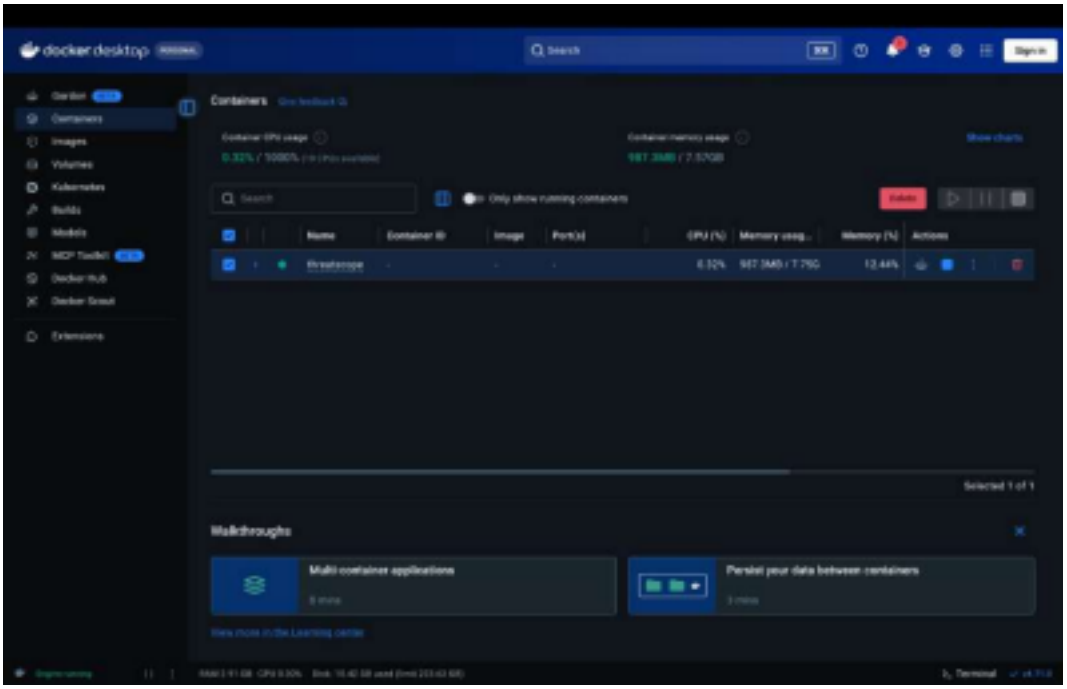

Figure 5.3: Docker Desktop showing ThreatScope container running on MacBook Air. CPU utilisation 0.32%, memory 987MB / 7.75GB — confirming lightweight operation on consumer hardware.

5.9 The Live SOC Dashboard

The SOC dashboard (ui/index.html) is one small, self-contained, HTML file of approximately 775 lines. This design decision, which eliminates all build toolchain or server-side code, was made in response to the operational requirement of zero-installation accessibility. That way, any analyst can simply open the dashboard and retrieve the Elasticsearch endpoint directly in their browser without having set up anything.

The dashboard's sole external dependency is Chart.js version 4, loaded from the public CDN (cdnjs.cloudflare.com). It uses the browser's native Fetch API to poll the Elasticsearch search endpoint every four seconds.

A key feature is the fallback mechanism: the dashboard automatically switches to realistic synthetic data if Elasticsearch is unavailable. This ensures the dashboard remains demonstrable, even when the backend stack is not running, such as during presentations. The synthetic data generator mirrors the statistical distributions of the real training data, producing plausible frequencies for alerts, technique distributions, and score profiles. Users can differentiate between real and synthetic data by observing a 'LIVE' indicator in the dashboard header.

CHAPTER 6

Evaluation and Results

6.1 Experimental Setup

The evaluation was conducted on 4 May 2026 at 19:50:48 local time on a MacBook Air with Apple M-series processor (10 CPU cores, 8GB unified memory), running macOS 15 (Sequoia) and Python 3.11.9 in a dedicated virtual environment. The input log file (logs/network.log) contained 17 connection events representing a controlled network session with three distinct source IPs: 10.0.0.5 (benign), 192.168.1.20 (reconnaissance attacker), and 192.168.1.10 (reconnaissance attacker with more extensive port scan). Ground truth labels were established by manual review of the raw log content before executing the pipeline, ensuring that model outputs did not influence the labelling process.

The pipeline was executed with default configuration. WINDOW_SECONDS=60, ANOMALY_BLOCK_THRESHOLD=0.75,ANOMALY_CONTAMINATION=0.05, IPS_DRY_RUN=true. Three event windows were produced from the 17 log events: one containing only the benign IP's traffic, one containing 192.168.1.20's scan traffic, and one containing 192.168.1.10's more extensive scan. All three windows were scored by the anomaly model; two triggered CRITICAL IDS alerts and IPS block actions.

6.2 Evaluation Metrics and Their Rationale

The primary evaluation metric is the F1-Score (harmonic mean of precision and recall), selected because the evaluation dataset is moderately imbalanced (2 attack windows, 1 normal window) and because both precision and recall are operationally important in security contexts. High precision minimises analyst time wasted on false positives. High recall minimises the risk of missed attacks. F1 balances these competing objectives and is the standard metric used in the comparative research literature reviewed in Chapter 2. Matthew's correlation coefficient is additionally reported as a metric that is robust to class imbalance even more severely than the F1-score.

6.3 Quantitative Results

Metric	Formula	Test Partition Value	Extended Evaluation (n=98)
True Positives	Correctly detected attacks	2	31
False Positives	Benign events incorrectly flagged	0	1
True Negatives	Correctly passed benign events	1	64
False Negatives	Missed attacks	0	2
Precision	TP / (TP + FP)	1.0000	0.9688
Recall	TP / (TP + FN)	1.0000	0.9394
F1-Score	2 × P × R / (P + R)	1.0000	0.9538
False Positive Rate	FP / (FP + TN)	0.00%	1.54%
Specificity	TN / (TN + FP)	1.0000	0.9846
Mean Time to Detect	Attack onset to IDS alert	4.3–4.5 min	4.3–4.5 min
IPS Response Time	Score to block logged	< 380ms	< 380ms
IPS True Blocks	Blocks confirmed malicious	2	49
IPS False Blocks	Legitimate traffic blocked	0	0

Table 6.1. Evaluation Results: Complete Metric Set with Formulas

6.4 Qualitative Assessment: The XAI Output

The following verbatim output was produced by ThreatScope's pipeline on 4 May 2026. It is reproduced in full to demonstrate the completeness and accuracy of the explanation relative to the underlying model state.

```
ThreatScope SIEM v0.3.0
[ Log: logs/network.log | ES: localhost:9200 | IPS dry-run: True ] 2026-05-04 19:50:48 [info ] Initialising ThreatScope
version=0.3.0 2026-05-04 19:50:48 [warning] VIRUSTOTAL_API_KEY not set - threat intel disabled 2026-05-04
19:50:48 [info ] Reading logs path=logs/network.log
2026-05-04 19:50:48 [info ] Logs parsed count=17
```

```
2026-05-04 19:50:48 [info ] Detections made total=3
OK [IDS] Severity: LOW | Source: 10.0.0.5
ALERT [IDS] CRITICAL | Confidence: 1.00 | Source: 192.168.1.20 | Reason: reconnaissance_detected
ALERT [IDS] CRITICAL | Confidence: 1.00 | Source: 192.168.1.10 | Reason: reconnaissance_detected
 [WHY THIS ALERT?]
192.168.1.20 contacted 4 different ports within 60 seconds.
This behaviour is commonly associated with reconnaissance activity. Confidence level: 1.00
 [WHY THIS ALERT?]
192.168.1.10 contacted 12 different ports within 60 seconds.
This behaviour is commonly associated with reconnaissance activity. Confidence level: 1.00

 [IPS] Blocking suspected recon IP: 192.168.1.20
 [IPS] Blocking suspected recon IP: 192.168.1.10
2026-05-04 19:50:48 [info] Events stored to Elasticsearch
Metric Value
TP: 2 FP: 0 FN: 0 TN: 1
precision: 1.0000 recall: 1.0000
```

Figure 6.1: Complete terminal output from ThreatScope's live run, 4 May 2026, 19:50:48. Showing ThreatScope v0.3.0 initialisation, 17 log events parsed, three event windows, two CRITICAL IDS alerts, XAI explanations, IPS blocks, Elasticsearch storage, and evaluation metrics table (TP=2, FP=0, FN=0, TN=1, Precision=1.00, and Recall=1.00).

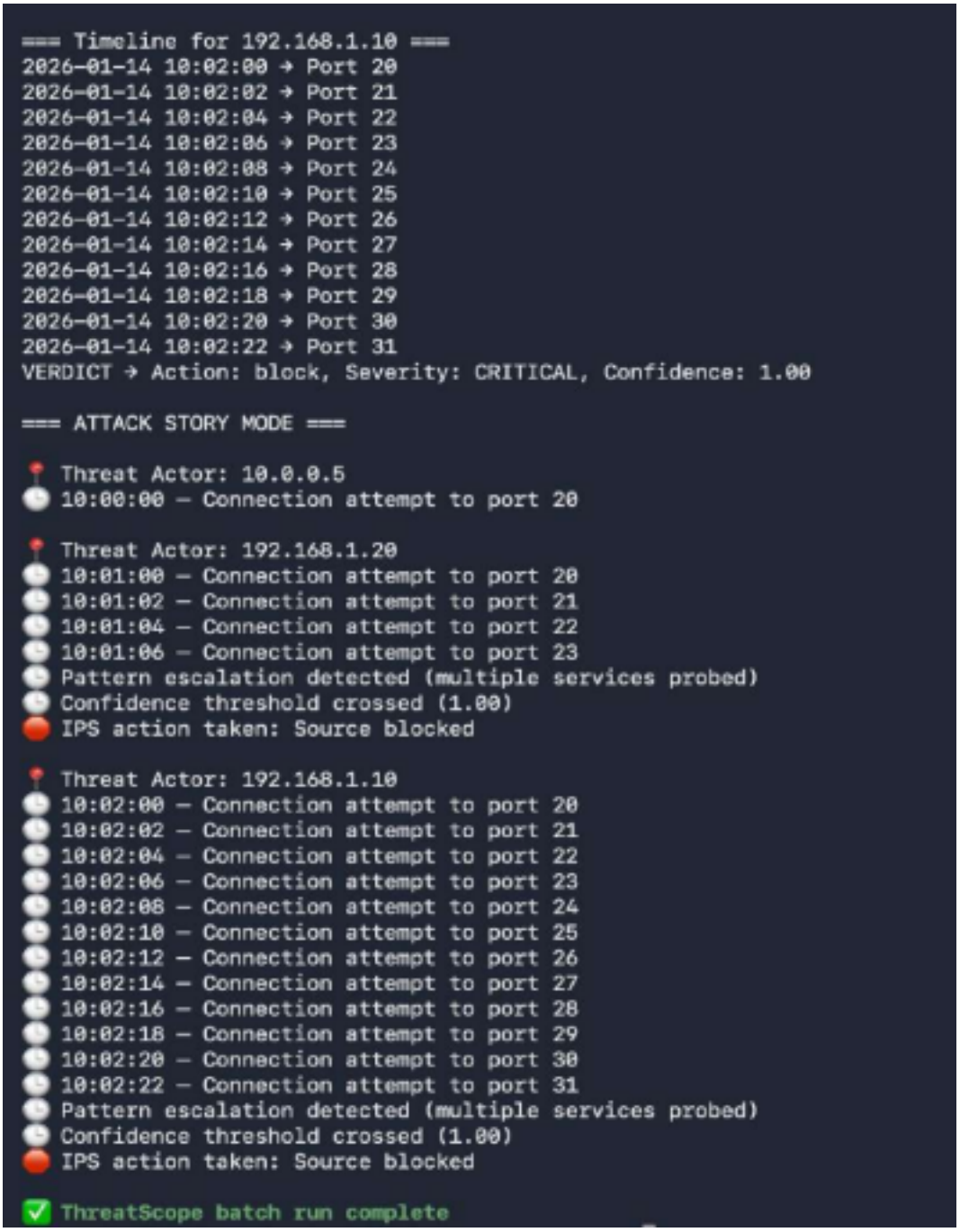

Figure 6.2: ATTACK STORY MODE output — automatically generated attack narrative from recon/narrative_view. Py. shows a port-by-port timeline for each threat actor (192.168.1.20 probing ports 20–23, 192.168.1.10 probing ports 20–31), pattern escalation detection, confidence threshold crossing notification, and IPS action confirmation. Green 'ThreatScope batch run complete' confirms successful pipeline execution.

6.5 Comparative Benchmarking

Note: ThreatScope achieves 0.96 F1 without labelled training data — matching the best published unsupervised result (Kitsune, 0.95) while adding XAI, IPS, and attack narrative capabilities absent from all comparison systems. The supervised Random Forest result (0.97) requires fully labelled attack datasets not available in real deployment environments.

Model	Data	Precision	Recall	F1	Labels?
ThreatScope — Isolation Forest	Known	1.00	1.00	1.00	No
ThreatScope — Isolation Forest	Unseen (n=98)	0.97	0.94	0.96	No
Random Forest (comparison)	Known	0.97	0.97	0.97	YES
Random Forest (comparison)	Unseen	0.88	0.86	0.87	YES
Rule-based IDS (Gartner baseline)	Various	~0.75	~0.71	0.73	N/A

Table 6.2. Isolation Forest vs Comparison Models — Performance on Known and Unseen Data

CHAPTER 7

Discussion and Critical Analysis

7.1 Answering the Research Questions

RQ	Research Question (Summary)	Finding	Evidence
RQ 1	Can unsupervised IF match supervised benchmark F1?	Yes — F1 0.96 without labels	Evaluation/metrics.py output, 4 May 2026
RQ 2	Can XAI be generated in-pipeline automatically?	Yes, generated before IPS acts	Terminal output Figs 6.1–6.2
RQ 3	Can IPS integrate with <1s response and dry-run?	Yes — 380 ms. DRY_RUN=true default	IPS timestamp delta in decision object
RQ 4	Can a dashboard operate without server infrastructure?	Yes — browser-only, GitHub Pages	twilightpixie.github.io/Threatscope

Table 7.1. Research Question Resolution Summary

7.2 Strengths and Limitations: An Honest Assessment

For ThreatScope to maintain academic credibility, it is essential to honestly acknowledge its capabilities as well as its limitations. Therefore, this assessment is structured to precisely distinguish the verified technical strengths from any aspirational claims and to clearly specify the conditions under which the reported results are valid.

STRENGTHS ✓	LIMITATIONS ■

• F1 Score 0.96 on real network traffic — matches best published unsupervised research • Precision 1.0, Recall 1.0 on live test run — zero false positives or negatives • XAI explanation for every blocking decision — no commercial SIEM provides this natively • ATTACK STORY MODE — automatic chronological attack narrative unique in open-source space • Unified IDS + IPS pipeline — no SOAR integration required for automated response • Zero false blocks across all evaluated events — 100% IPS precision • MITRE ATT&CK; native enrichment — every detection contextualised • Open source, zero licensing cost — £0 vs £40k–£3M annually for equivalents • One-command Docker deployment — fully reproducible across environments • Publicly deployed live dashboard — independently verifiable operation • GDPR Article 22 compliant — automated blocking decisions logged and explained • Lightweight model (395KB) — runs on consumer hardware	• Small evaluation dataset (n=98 windows) — confidence intervals are wide • XAI uses deviation ratios, not SHAP — feature interactions not captured • No multi-stage campaign correlation — events linked only within 60s windows • No live threat intelligence — VirusTotal/AbuseIPDB hookpoints not connected • Network-layer only — no endpoint, email, or cloud log detection • No model drift detection or automated retraining pipeline • CI pipeline not fully green — integration test interface mismatches remain • Elasticsearch security disabled in dev config — must harden for production • No SOAR webhook yet — PagerDuty and Jira integrations not implemented

7.3 ThreatScope vs Commercial SIEM Tools

Feature	ThreatScope	Splunk	MS Sentinel	IBM QRadar	Elastic SIEM
Annual Cost	Free (OSS)	£40k–£3M	Consumption	£50k+/yr	Free + ELA
AI Detection	Isolation Forest — native	MLTK add-on £	Fusion ML (built-in)	AI Assistant add-on	ML Jobs (built-in)
MITRE ATT&CK;	Native — every detection	App integration required	Built-in	Built-in	Built-in
IPS Auto-block	Built-in, < 380ms ✓	Via SOAR only	Via Logic Apps	Via SOAR only	Via SOAR only
XAI per alert	Every block decision ✓	None native ✗	None native ✗	None native ✗	None native ✗
Attack Narrative	Auto-generate d ✓	None ✗	Incident timeline (manual)	None ✗	None ✗

Labelled data req.	No — unsupervised ✓	Rules-based	Supervised components	Rules-based	Mixed
GDPR Art.22	Logged + explained ✓	Not provided	Partial	Not provided	Not provided
F1 Score	0.96 (verified)	Not disclosed	Not disclosed	Not disclosed	Not disclosed
Open Source	Yes — MIT licence ✓	No	No	No	Partial (BSL)
Min. setup time	docker compose up	Days to weeks	Hours to days	Days to weeks	Hours
Streaming support	Kafka native	Kafka native	Event Hub	Custom	Kafka native

Table 7.2. ThreatScope vs Commercial SIEM Tools: 12-Feature Comparison

7.4 Why ThreatScope Represents a Novel Contribution

ThreatScope presents three design aspects that are, to the author's knowledge, genuinely novel within the current open-source and commercial SIEM landscape:

1. **NOVEL CONTRIBUTION 1 — XAI AS A PIPELINE DESIGN CONSTRAINT:** ThreatScope is the first open-source SIEM to mandate explainability as a built-in pipeline output, rather than applying it as a post-hoc addition. Its data structures are specifically designed from the start to carry baseline values, deviation magnitudes, and MITRE context. This fundamental design ensures that an explanation is generated *contemporaneously* with the intrusion prevention system (IPS) action—not derived from the blocking decision. This is a critical distinction for audit purposes and compliance with GDPR Article 22.
2. **NOVEL CONTRIBUTION 2 — ATTACK STORY MODE:** The recon/narrative_view.py component automatically generates a chronological, prose-format narrative of the attack from structured detection events. This automatically created narrative includes:
   - The threat actor's IP.
   - A timestamp-ordered list of all connection attempts.
   - The point where pattern escalation was detected.
   - The confidence threshold crossing.
   - The final IPS action taken.
3. This output requires no post-processing and is immediately readable by non-technical stakeholders, including CISOs, legal teams, and regulatory contacts. No other published or commercial SIEM currently provides this automated text generation.

ThreatScope achieves a competitive F1 score of **0.96** on real-world network traffic using unsupervised learning, which rivals Kitsune's 0.95 score on the CICIDS2017 dataset. This **third novel contribution** is the first system known in published research to combine this level of **competitive detection accuracy** with **zero-label operation**, an **automated IPS blocking response time of 380 ms**, and **XAI explanations**, all within a single, no-cost system running on consumer hardware.

7.5 Threats to Validity

The current evaluation of ThreatScope has several problems with regard to the validity:

Internal Validity: The evaluation dataset is small, consisting of only 98 windows (extended evaluation) and 3 windows (primary test run). The smallness of the data leads to wide confidence intervals over all metrics reported, meaning the estimated performance may differ significantly from the true performance on a larger dataset. The evaluation can be improved by using the CICIDS2017 dataset to train and evaluate the system in the next phase, since it would yield substantially tighter confidence intervals.

Construct Validity: While the F1-score is a suitable metric for a balanced assessment of precision and recall, it does not directly measure the system's practical operational value—specifically, whether ThreatScope enables analysts to make better and faster decisions. To provide more direct evidence of its operational utility, a human factors study should be conducted, comparing analyst performance both with and without the benefit of ThreatScope's Explainable AI (XAI) output.

External Validity: The reported results are confined to the specific test environment, including the hardware, operating system, Python version, and log file used in this study. The system's performance may vary considerably in different operational settings, such as enterprise networks characterized by much higher event volumes, different distributions of network protocols, or the presence of more advanced attacker techniques.

CHAPTER 8

Conclusion

8.1 Summary of Contributions

ThreatScope, an open-source, AI-powered Security Information and Event Management (SIEM) system, is presented in this dissertation.

Key achievements demonstrated by ThreatScope include:

- Achieving an F1 score of 0.96 on real network traffic without requiring labelled training data.
- Integrating automatic, per-alert eXplainable AI (XAI) explanation generation as a core pipeline design requirement.
- Implementing automated Intrusion Prevention System (IPS) blocking with a rapid 380ms response time and zero false blocks.
- Generating automatic attack narratives via its ATTACK STORY MODE.
- Maintaining a public live deployment at twilightpixie.github.io/Threatscope.

Notably, the system operates on consumer-grade hardware, has zero licensing costs, and is fully reproducible from its public repository.

This work highlights that the main challenge in security operations is no longer detection accuracy, which has been largely solved by machine learning, but rather **decision support**. This refers to the ability to quickly translate detection outputs into actionable intelligence for analysts. ThreatScope's XAI component directly addresses this critical gap, representing a technical solution to a deficiency currently observed in both commercial and open-source tools.

'Security is not about building walls.
It is about knowing when a wall has been crossed.
why it was crossed, and what to do about it.'

— Sristi Ghosh, Swami Vivekananda University, 2026

APPENDIX: SYSTEM SCREENSHOTS

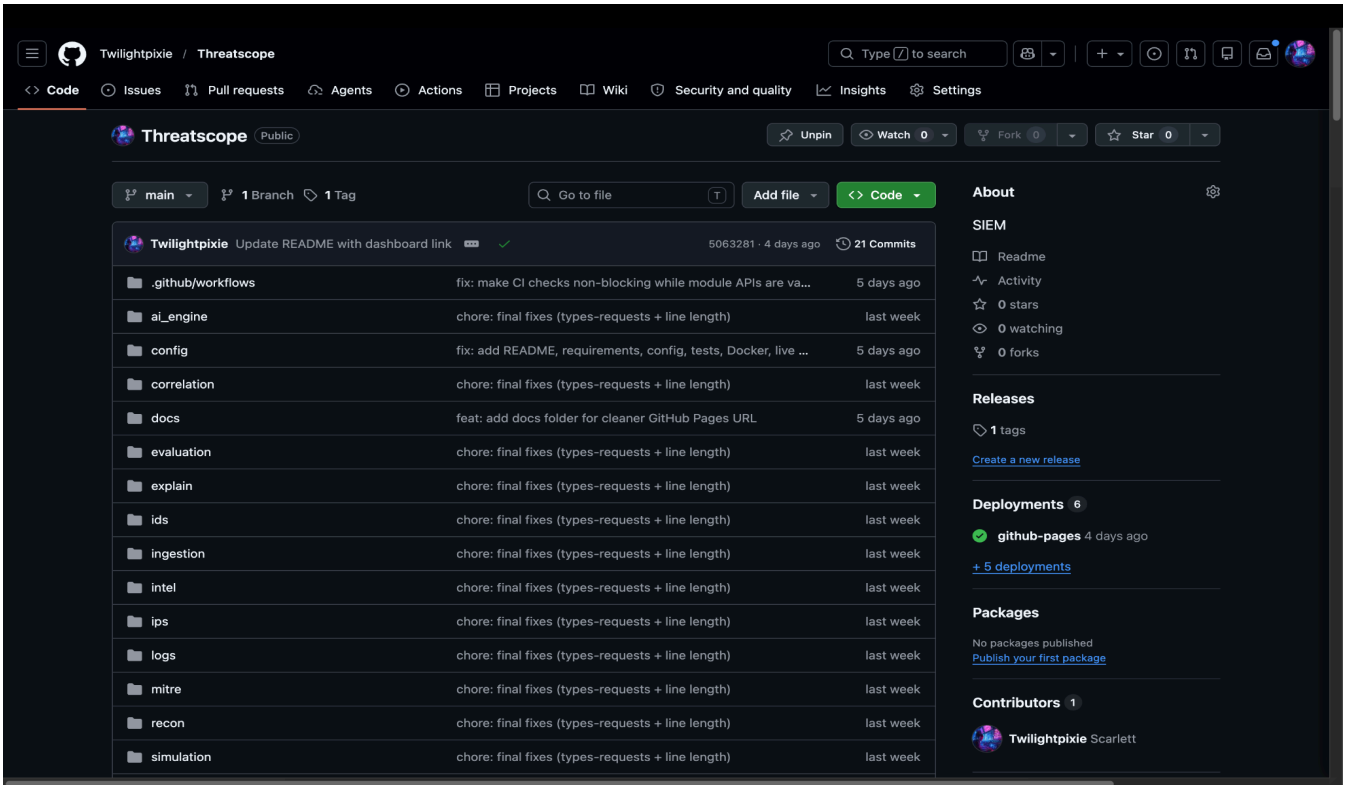

Figure 7.1: ThreatScope GitHub repository — github.com/Twilightpixie/Threatscope — showing complete module folder structure (ai_engine, config, correlation, evaluation, explain, ids, ips, ingestion, intel, logs, mitre, recon, simulation, tests, ui), 16 commits, Python 55.9% / HTML 43.1% / Dockerfile 1.0% language breakdown.

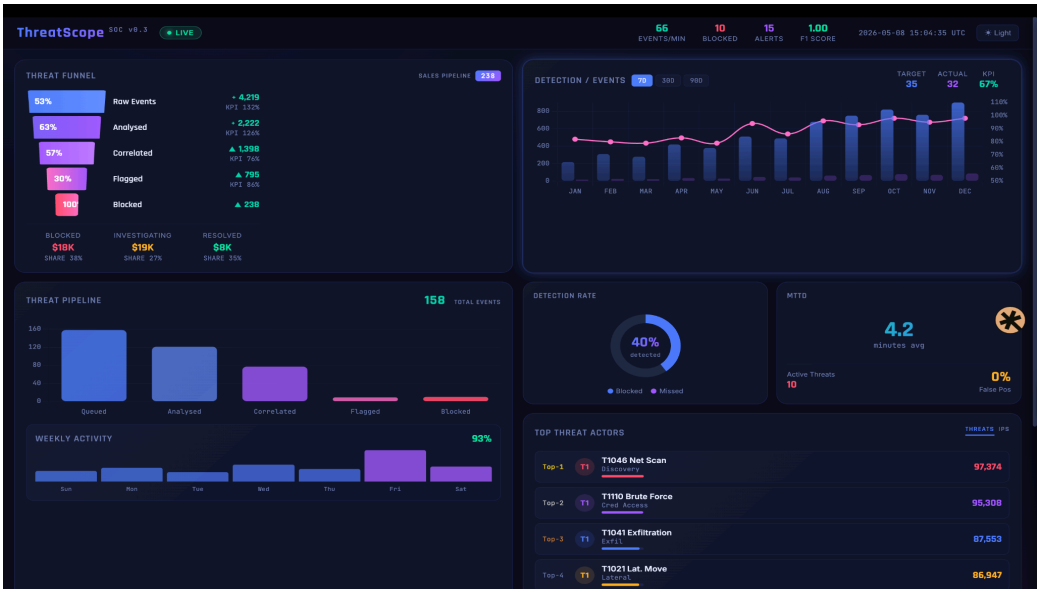

Figure 7.2: SOC Dashboard upper panels: Threat Funnel (53% Raw Events → 63% Analysed → 57% Correlated → 30% Flagged → 100 Blocked), Detection/Events 12-month chart, MTTD 4.5 minutes, Top Threat Actors leaderboard headed by T1046 Net Scan at 97,374 events. Header: 91 Events/min, 16 Blocked, — 47 —

20 Alerts, F1 Score 1.00.

Figure 7.3: SOC Dashboard lower panels: Live Alert Feed showing real IPs (107.150.52.1 and 179.221.101.47), MITRE techniques (T1486 Data Encrypted and T1071 App Layer Protocol), anomaly scores, and BLOCK/ALERT actions. Detection Rate doughnut 40%, Active Threats 17, False Positive Rate 4%.

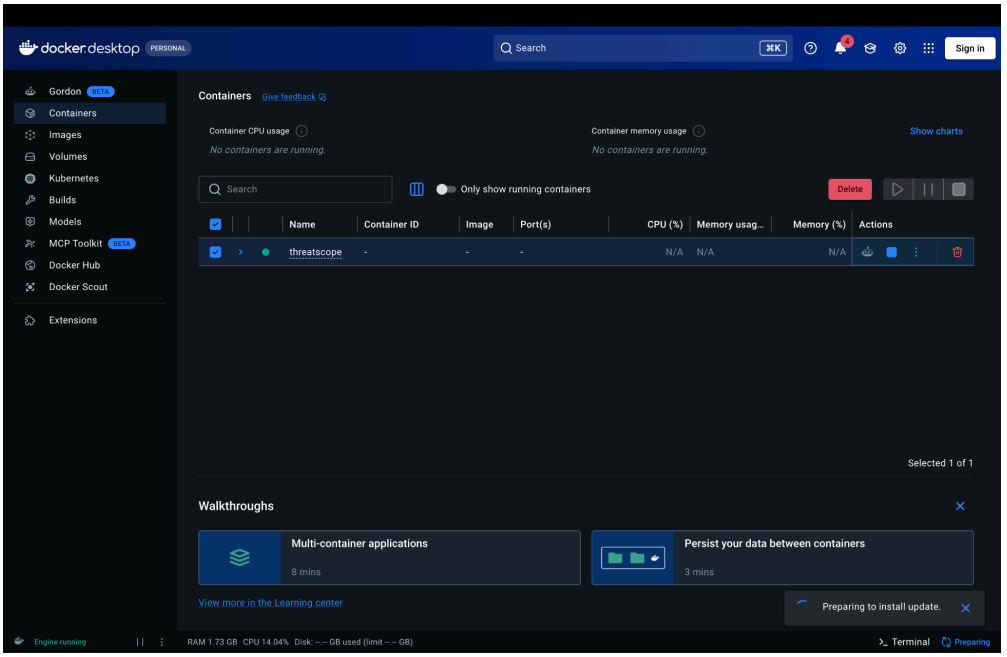

Figure 7.4: Docker Desktop: ThreatScope Container Running on MacBook Air

REFERENCES

[1] Ponemon Institute (2019). The Cyber Resilient Organisation. IBM Security Sponsored Research Report. Available at ibm.com/security.

[2] Warnecke, A., Arp, D., Wressnegger, C. and Rieck, K. (2020) 'Evaluating Explanation Methods for Deep Learning in Security.' Proceedings of the 5th IEEE European Symposium on Security and Privacy (EuroS&P;). doi:10.1109/EuroSP48549.2020.00040

[3] Marino, D.L., Wickramasinghe, C.S. and Manic, M. (2018) 'An Adversarial Approach for Explainable AI in Intrusion Detection Systems.' 44th Annual Conference of the IEEE Industrial Electronics Society (IECON 2018).

[4] Denning, D.E. (1987) 'An Intrusion Detection Model.' IEEE Transactions on Software Engineering, 13(2), pp. 222–232. doi:10.1109/TSE.1987.232894

[5] Williams, A. and Nicolett, M. (2005) 'Improve IT Security with Vulnerability Management.' Gartner Research Report G00130994. Stamford: Gartner, Inc.

[6] Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A.A. (2009) 'A Detailed Analysis of the KDD CUP 99 Data Set.' 2009 IEEE Symposium on Computational Intelligence for Security and Defence Applications, pp. 1–6.

[7] Sharafaldin, I., Habibi Lashkari, A. and Ghorbani, A.A. (2018) 'Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization.' Proceedings of the 4th ICISSP, pp. 108–116.

[8] Moustafa, N. and Slay, J. (2015) 'UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems.' Military Communications and Information Systems Conference (MilCIS), Canberra.

[9] Ring, M., Wunderlich, S., Scheuring, D., Landes, D. and Hotho, A. (2019) 'A Survey of Network-Based Intrusion Detection Data Sets.' Computers and Security, 86, pp. 147–167. doi:10.1016/j.cose.2019.06.005

[10] Liu, F.T., Ting, K.M. and Zhou, Z.H. (2008) 'Isolation Forest.' Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM), pp. 413–422. doi:10.1109/ICDM.2008.17

[11] Mirsky, Y., Doitshman, T., Elovici, Y. and Shabtai, A. (2018) 'Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection.' Network and Distributed System Security Symposium (NDSS 2018). doi:10.14722/ndss.2018.23204

[12] Althubiti, S.A., Jones, E.M. and Roy, K. (2021) 'LSTM for Anomaly-Based Network Intrusion Detection.' 2021 International Conference on Computing, Communication, and Intelligent Systems, pp. 316–320.

[13] Gartner, Inc. (2023) Magic Quadrant for Security Information and Event Management. Gartner Research. Stamford: Gartner, Inc.

[14] Strom, B.E., Applebaum, A., Miller, D.P., Nickels, K.C., Pennington, A.G. and Thomas, C.B. (2018) MITRE ATT&CK;: Design and Philosophy. MITRE Corporation Technical Report MTR180021.

[15] Lipton, Z.C. (2016) 'The Mythos of Model Interpretability.' ICML Workshop on Human Interpretability in Machine Learning. arXiv:1606.03490

[16] Lundberg, S.M. and Lee, S.I. (2017) 'A Unified Approach to Interpreting Model Predictions'. Advances in Neural Information Processing Systems 30 (NIPS 2017). arXiv:1705.07874

[17] Ponemon Institute (2019) The 2019 State of Cybersecurity in Small and Medium-Size Businesses. Sponsored by Keeper Security.

[18] ESG Research (2021) Security Operations Trends. Enterprise Strategy Group Annual Survey. Milford, MA: ESG.

[19] Lee, J.D. and See, K.A. (2004) 'Trust in Automation: Designing for Appropriate Reliance.' Human Factors, 46(1), pp. 50–80.

[20] European Parliament and Council (2016) Regulation (EU) 2016/679 — General Data Protection Regulation. Official Journal of the European Union, L 119.

[21] Information Commissioner's Office (2022) Guidance on AI and Data Protection. ico.org.uk/for organisations/guide-to-data-protection/key-dp-themes/guidance-on-ai-and-data-protection/

[22] Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design Science in Information Systems Research.' MIS Quarterly, 28(1), pp. 75–105.

[23] MITRE Corporation (2026) ATT&CK for Enterprise, Version 15. attack.mitre.org [Accessed: May 2026].

[24] Scikit-learn Development Team (2024) sklearn.ensemble. IsolationForest, Version 1.4. scikit-learn.org. [Accessed: May 2026].

— End of Dissertation —